

۱ ساختار کلی پروژه

هدف کلی الگوریتم، تقسیم یک تصویر به چند ناحیه‌ی همگن است. برای این منظور تصویر را به صورت یک گراف مدل می‌کنیم که در آن هر پیکسل یک گره است و پیکسل‌های شبیه و نزدیک با یالی سنگین به هم متصل می‌شوند. سپس با استفاده از مقادیر و بردارهای ویژه‌ی لاپلاسیان این گراف، مرزهای کم‌هزینه‌ی برش شناسایی و تصویر از همان نقاط قطعه‌بندی می‌شود. برای خوانایی و ماژولار بودن پیاده‌سازی، این مسیر به هشت فایل مستقل تقسیم شده است؛ هر فایل یک گام از زنجیره را انجام می‌دهد و فایل `run_all.py` همه را به ترتیب اجرا کرده و شکل‌های خروجی را در پوشه‌ی `outputs/` ذخیره می‌کند.

روند کلی داده در خطلوله: تصویر $\rightarrow$ ویژگی‌های هر پیکسل $(X, F) \rightarrow$ ماتریس تشابه $W \rightarrow$ لاپلاسیان $L_{sym} \rightarrow$ بردارهای ویژه‌ی کوچک $(z_2, \dots, z_k) \rightarrow$ خوشه‌بندی k -means $\rightarrow$ برچسب‌هر پیکسل (تصویر قطعه‌بندی‌شده). ورودی هر گام، خروجی گام پیش از آن است.

فایل	وظیفه
<code>step1_image.py</code>	بارگذاری تصویر و ساخت بردارهای ویژگی (مکان و روشنایی)
<code>step2_weights.py</code>	ساخت ماتریس تشابه متقارن W (پرسش ۱)
<code>step3_laplacian.py</code>	ساخت لاپلاسیان نرمال‌شده L_{sym} و بررسی طیف (پرسش ۲)
<code>step5_plu.py</code>	تجزیه‌ی PLU و حل دستگاه مثلثی از پایه (پرسش ۵)
<code>step4_deflation.py</code>	توان معکوس + حذف برای یافتن بردارهای فیدلر (پرسش ۳ و ۴)
<code>step6_segmentation.py</code>	k -means دستی و قطعه‌بندی برای $k = 2, 3, 4$ (پرسش ۶)
<code>step7_lanczos.py</code>	لانتساش روی ماتریس $\hat{X}$ تصویر بزرگ (پرسش ۷)
<code>step8_zebra.py</code>	اجرای همان خطلوله روی تصویر واقعی گورخر (نمونه‌ی مقایسه)

۲ گام ۱ - بارگذاری تصویر و ساخت ویژگی‌ها

معیار شباهت دو پیکسل بر دو عامل استوار است: میزان هم‌رنگی و میزان نزدیکی مکانی. بنابراین برای هر پیکسل دو ویژگی نگه می‌داریم: مختصات مکانی‌اش $X_i = (row, col)$ و شدت روشنایی‌اش F_i . برای آنکه بتوان درستی الگوریتم را به سادگی ارزیابی کرد، به جای یک تصویر واقعی ابتدا یک تصویر مصنوعی ساده تولید می‌شود: تابع `make_synthetic_image` یک دیسک روشن (شدت ≈ 0.85) را در مرکز یک زمینه‌ی تیره (شدت ≈ 0.15) قرار می‌دهد و نویز گاوسی کوچکی به آن می‌افزاید. از آنجا که پاسخ درست (تفکیک دایره از زمینه) از پیش معلوم است، صحت خروجی به سادگی قابل دآوری است.

روند کد. تصویر یک آرایه‌ی دوبعدی $H \times W$ است، اما برای کار با گراف لازم است هر پیکسل به صورت یک سطر بازنامایی شود. تابع `build_features` این تبدیل را انجام می‌دهد: نخست با `reshape` تصویر را به شکلی درمی‌آورد که هر پیکسل یک سطر از ماتریس روشنایی F باشد (در نتیجه $n = H \cdot W$ سطر خواهیم داشت)؛ سپس با `np.mgrid` شماره‌ی سطر و ستون هر پیکسل را می‌سازد و آن‌ها را در ماتریس مکان X می‌چیند. خروجی این تابع، همین دو ماتریس X و F است که ورودی گام بعد (ساخت ماتریس تشابه) خواهند بود.

```

1 def build_features(img):
2     # X: pixel coords (n, 2), F: intensities (n, c)
3     H, W = img.shape[:2]
4     F = img.reshape(H * W, -1)           # each pixel -> one feature row
5     rows, cols = np.mgrid[0:H, 0:W]
6     X = np.stack([rows.ravel(), cols.ravel()], axis=1).astype(float)
7     return X, F, (H, W)

```

خروجی اجرا:

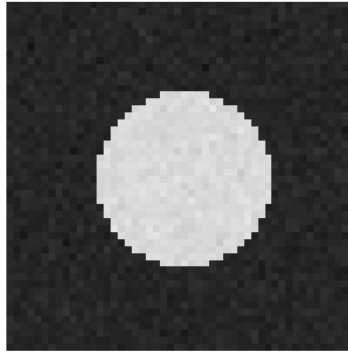
```

image (50, 50) -> n = 2500 pixels, X (2500, 2), F (2500, 1)
intensity range: [0.072, 0.902]

```

یعنی تصویر 50×50 به $n = 2500$ گره تبدیل شد، ماتریس مکان X ابعاد $(2500, 2)$ و ماتریس روشنایی F ابعاد $(2500, 1)$ دارد. بازه‌ی شدت $[0.072, 0.902]$ است - کف و سقف حول همان 0.15 و 0.85 به علاوه نویز، که نشان می‌دهد تصویر درست ساخته شده است.

prepared image 50x50



خروجی تصویری گام ۱: دیسک روشن روی زمینه تیره (50 × 50).

۳ گام ۲ – ماتریس تشابه W (پرسش ۱)

پس از استخراج ویژگی‌ها، گراف ساخته می‌شود. برای هر جفت پیکسل یک وزن تعریف می‌کنیم که میزان شباهت آن‌ها را نشان می‌دهد: هرچه دو پیکسل هم‌رنگ‌تر و نزدیک‌تر باشند وزن بزرگ‌تر است، و با افزایش اختلاف روشنایی یا فاصله، وزن به سرعت کاهش می‌یابد. یک ملاحظه‌ی کارایی نیز در نظر گرفته شده است: ارتباط هر پیکسل با پیکسل‌های بسیار دور اهمیتی ندارد، بنابراین برای هر جفت که فاصله‌ی مکانی‌شان از شعاع r بیشتر باشد وزن را صفر قرار می‌دهیم. در نتیجه بیشتر درایه‌های W صفر می‌شوند (ماتریس تُنک می‌شود) و در حافظه و زمان محاسبات صرفه‌جویی چشمگیری حاصل می‌شود.

```
1 def build_weight_matrix(X, F, sigma_I=0.1, sigma_X=4.0, r=5.0, dense=True):
2     sqX = np.sum(X ** 2, axis=1)
3     dX2 = sqX[:, None] + sqX[None, :] - 2.0 * (X @ X.T)
4     sqF = np.sum(F ** 2, axis=1)
5     dF2 = sqF[:, None] + sqF[None, :] - 2.0 * (F @ F.T)
6     np.clip(dX2, 0.0, None, out=dX2) # rounding can make tiny negatives
7
8     W = np.exp(-dF2 / sigma_I ** 2) * np.exp(-dX2 / sigma_X ** 2)
9     W[dX2 >= r ** 2] = 0.0 # keep only neighbors within r
10    np.fill_diagonal(W, 0.0) # W_ii = 0
11    return W if dense else sp.csr_matrix(W)
```

روند کد. محاسبه‌ی مستقیم فاصله‌ها با دو حلقه‌ی تودرتو برای $n = 2500$ گره به میلیون‌ها عملیات می‌انجامد و بسیار کند است. به جای آن از محاسبه‌ی برداری استفاده می‌کنیم: با اتحاد $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a \cdot b$ می‌توان تمام فاصله‌های دوبه‌دو را یک جا و با ضرب ماتریسی به دست آورد. بدین ترتیب کد نخست $dX2$ (مربع فاصله‌های مکانی) و $dF2$ (مربع فاصله‌های روشنایی) را می‌سازد و سپس دو کرنل گاوسی را در هم ضرب می‌کند تا W حاصل شود. سه نکته‌ی پیاده‌سازی که در کد لحاظ شده است:

- گاهی بر اثر خطای گردکردن، مربع فاصله مقداری منفی بسیار کوچک می‌شود (که از نظر ریاضی ناممکن است)؛ `clip` این مقادیر را به صفر می‌برد تا جذر و تابع نمایی دچار خطا نشوند.
- خط `W[dX2 >= r**2] = 0` شرط همسایگی را اعمال می‌کند: هر جفت با فاصله‌ی بیش از r صفر می‌شود. این خط عامل اصلی تُنک ماندن W است.

• `np.fill_diagonal(W, 0)` قطر را صفر می‌کند، زیرا هیچ پیکسلی با خودش یال ندارد ($W_{ii} = 0$). شایان ذکر است که تقارن W به صورت ذاتی برقرار است و نیازی به تحمیل آن نیست: چون $dX2$ و $dF2$ از رابطه‌ای متقارن نسبت به i و j ساخته شده‌اند، W_{ji} و W_{ij} یکسان به دست می‌آیند – همان نتیجه‌ای که در پرسش ۱ به صورت تحلیلی اثبات شد.

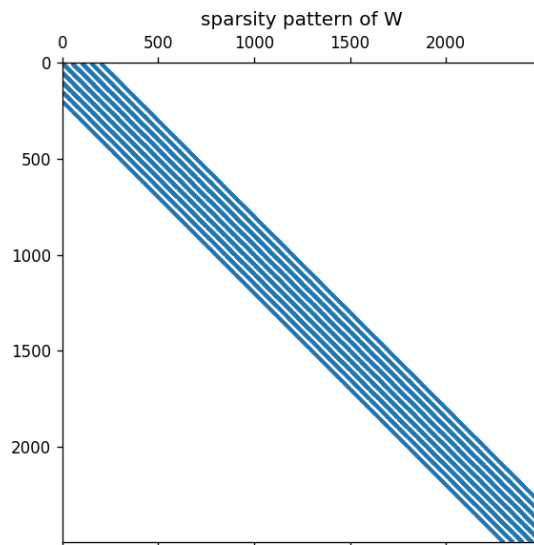
```
1 def symmetry_error(W):
2     return float(np.linalg.norm(W - W.T, ord="fro")) # ||W - W^T||_F
```

خروجی اجرا:

```
params: sigma_I=0.1, sigma_X=4.0, r=5.0
W shape : (2500, 2500), range [0.0000, 0.9394]
max|diag|: 0.00e+00
nonzeros: 2.51%
||W - W^T||_F = 0.000e+00
```

پاسخ عددی پرسش. پرسش ۱-ب می‌خواست تقارن W را عددی تأیید کنیم. تابع `symmetry_error` کمیت $\|W - W^T\|_F$ را حساب می‌کند و خروجی دقیقاً 0.000×10^0 شد. یعنی نه فقط تقریباً، بلکه بیت‌به‌بیت $W_{ij} = W_{ji}$ ؛ این تأیید عملی همان اثبات تحلیلی است. همچنین بیشینه قطر $0 = \text{قرارداد } W_{ii} = 0$ (رعایت شده) و فقط 2.51% درایه‌ها ناصفرند، یعنی ماتریس به شدت تُنک است.

نمودار زیر (`step2_W_spy.png`) این تُنکی را به صورت یک نوار باریک قطری نشان می‌دهد: هر پیکسل فقط با همسایه‌های نزدیکش (که در ترتیب سطری-ستونی تصویر شماره‌های نزدیک دارند) پیوند دارد و بقیه‌ی ماتریس صفر است.



خروجی تصویری گام ۲: الگوی تُنکی W ؛ فقط یک نوار باریک قطری ناصفر است.

۴ گام ۳ – لاپلاسیان نرمال‌شده (پرسش ۲)

ماتریس W به تنهایی برای برش کافی نیست و باید نرمال‌سازی شود تا گره‌های پُرارتباط به‌طور نامتناسب غالب نشوند. ابتدا درجه‌ی هر گره $d_i = \sum_j W_{ij}$ ، یعنی مجموع وزن یال‌هایش محاسبه و سپس لاپلاسیان نرمال‌شده‌ی $L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$ ساخته می‌شود. تمام خواصی که در پرسش ۲ اثبات شد (تقارن، قرارگیری طیف در $[0, 2]$ و بردار ویژه‌ی بدیهی صفر) مربوط به همین ماتریس است، و بردارهای ویژه‌ی کوچک آن همان مؤلفه‌هایی هستند که در گام‌های بعد برای برش به کار می‌روند.

```
1 def build_normalized_laplacian(W):
2     W = np.asarray(W, dtype=float)
3     n = W.shape[0]
4     d = W.sum(axis=1)
5     d_inv_sqrt = 1.0 / np.sqrt(np.where(d > 0, d, 1.0)) # guard isolated nodes
6     L = np.eye(n) - d_inv_sqrt[:, None] * W * d_inv_sqrt[None, :]
7     L = 0.5 * (L + L.T) # enforce exact symmetry
8     return L, d, d_inv_sqrt
```

روند کد. کد گام‌به‌گام همان فرمول را می‌سازد. ابتدا با `W.sum(axis=1)` درجه‌ی همه‌ی گره‌ها یک‌جا محاسبه می‌شود. سپس به‌جای ساخت کامل ماتریس قطری $D^{-1/2}$ (که کم‌بار و پُرهزینه است)، تنها بردار `d_inv_sqrt` نگه‌داری و با یک ضرب سطری/ستونی اثر آن روی W اعمال می‌شود؛ نتیجه یکسان اما بسیار کم‌هزینه‌تر است. دو محافظ عددی نیز در نظر گرفته شده است: (۱) برای گره‌های منزوی که درجه‌ی صفر دارند، به‌جای تقسیم بر صفر مقدار ۱ قرار داده می‌شود؛ (۲) خط `L = 0.5 * (L + L.T)` تقارن را به صورت صریح تضمین می‌کند، زیرا خطاهای کوچک می‌توانند تقارن تحلیلی را اندکی مخدوش کنند.

تابع `spectral_report` بخشی از الگوریتم اصلی نیست و صرفاً برای راستی‌آزمایی افزوده شده است: با استفاده از تابع `eigh`، ادعاهای تئوری پرسش ۲ به صورت عددی سنجیده می‌شوند تا اطمینان حاصل شود که خروجی کد با نتایج اثبات‌شده مطابقت دارد:

```
1 evals, evects = np.linalg.eigh(L) # reference spectrum (verification only)
```

```

2 v = np.sqrt(d); v /= np.linalg.norm(v) # trivial eigenvector D^{1/2} 1
3 Lv = L @ v
4 "min_eig": evals.min(), "max_eig": evals.max(),
5 "in_0_2": evals.min() >= -1e-8 and evals.max() <= 2 + 1e-8,
6 "trivial_residual": np.linalg.norm(Lv), # ||L (D^{1/2} 1)||
7 "trivial_rayleigh": v @ Lv, # v^T L v
8 "num_near_zero": int(np.sum(evals < 1e-6)),

```

خروجی اجرا:

```

L_sym shape : (2500, 2500), ||L - L^T||_F = 0.000e+00
eig range = [-3.270764e-16, 1.096398e+00], all in [0, 2]? True
||L * (D^{1/2} 1)|| = 1.913e-16, Rayleigh = 2.754e-19
num near-zero eig = 2
trivial in bottom subspace = 1.000000

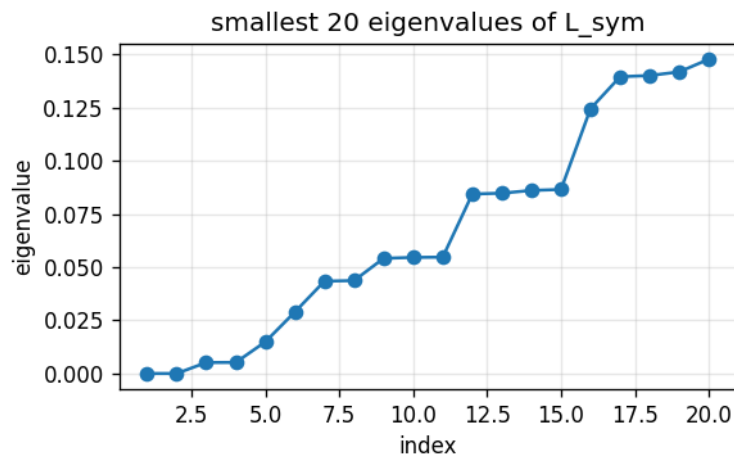
```

پاسخ عددی پرسش. پرسش ۲-الف (تقارن). خروجی $\|L - L^T\|_F = 0$ نشان می‌دهد L_{sym} دقیقاً متقارن است – تأیید عددی همان اثبات $L_{\text{sym}}^T = L_{\text{sym}}$.

پرسش ۲-ب (طیف در $[0, 2]$). کوچک‌ترین مقدار ویژه $-3.3 \times 10^{-16} \approx 0$ (عملاً صفر و فقط خطای ماشین) و بزرگ‌ترین $1.096 \approx$ پرچم `in_0_2` برابر True شد. یعنی کل طیف داخلی بازه‌ی تئوری $[0, 2]$ نشست.

پرسش ۲-ج (بردار بدیهی صفر). باقی‌مانده‌ی $\|L(D^{1/2}1)\| = 1.9 \times 10^{-16}$ و خارج‌قسمت ریلی $v^T Lv = 2.8 \times 10^{-19}$ (هر دو ≈ 0) عملاً معادله‌ی $L_{\text{sym}}(D^{1/2}1) = 0$ را تأیید می‌کنند؛ پس $D^{1/2}1$ واقعاً بردار ویژه‌ی مقدار ویژه‌ی صفر است.

مشاهده‌ی درخور توجه آن است که تعداد مقادیر ویژه‌ی نزدیک صفر برابر ۲ به‌دست آمد. این نشان می‌دهد که با این پارامترها، دیسک و زمینه چنان کم به هم متصل‌اند که گراف تقریباً به دو مؤلفه‌ی مجزا تفکیک می‌شود – که با انتظار اولیه (یک شیء و یک پس‌زمینه) سازگار است. در نمودار زیر ۲۰ مقدار ویژه‌ی کوچک ترسیم شده است؛ منحنی از صفر آغاز و به‌صورت پله‌ای صعود می‌کند، و محل جهش‌های ناگهانی (گاف طیفی) بیانگر تعداد خوشه‌های طبیعی تصویر است.



۲۰ مقدار ویژه‌ی کوچک L_{sym} ؛ صعودی و آغازشونده از صفر.

۵ گام ۵ – تجزیه‌ی PLU از پایه (پرسش ۵)

پیش از یافتن بردارهای ویژه، به یک ابزار پایه نیاز داریم: حل دستگاه‌های خطی. کار اصلی روش توان معکوس، حل مکرر دستگاه $(\mu I - L)x = b$ است. معکوس‌گیری کامل ماتریس هم پرهزینه و هم از نظر عددی نامطلوب است؛ رویکرد استاندارد بهتر آن است که ماتریس یک‌بار به حاصل‌ضرب دو ماتریس مثلثی تجزیه شود ($PA = LU$) و پس از آن، برای هر سمت راست تنها دو دستگاه مثلثی ساده حل شود. چون ماتریس سمت چپ در تمام تکرارها ثابت است و فقط بردار b تغییر می‌کند، این رویکرد صرفه‌جویی محاسباتی چشمگیری به‌همراه دارد.

```

1 def plu_decomposition(A):
2     # Gaussian elimination with partial pivoting -> P A = L U
3     U = np.array(A, dtype=float); n = U.shape[0]
4     L = np.eye(n); P = np.eye(n)

```

```

5 for k in range(n):
6     p = k + int(np.argmax(np.abs(U[k:, k]))) # largest pivot in column k
7     if p != k:
8         U[[k, p], :] = U[[p, k], :]
9         P[[k, p], :] = P[[p, k], :]
10        L[[k, p], :k] = L[[p, k], :k]
11        L[k + 1:, k] = U[k + 1:, k] / U[k, k] # multipliers
12        U[k + 1:, k:] -= np.outer(L[k + 1:, k], U[k, k:]) # eliminate below pivot
13 return P, L, U

```

روند ۳. این پیاده‌سازی همان حذف گاوسی با محورگیری جزئی است. کد ستون‌به‌ستون پیش می‌رود و در هر ستون: نخست با argmax بزرگ‌ترین درایه (به قدر مطلق) را می‌یابد و با تعویض سطر آن را روی قطر قرار می‌دهد — این کار از تقسیم بر مقادیر بسیار کوچک و بروز ناپایداری عددی جلوگیری می‌کند؛ همین تعویض روی P (که جابه‌جایی‌ها را ثبت می‌کند) و بخش ساخته‌شده‌ی L نیز اعمال می‌شود. سپس ضرایب حذف در L ذخیره و با یک به‌روزرسانی برداری (np.outer) تمام سطرهای زیر محور یک‌جا صفر می‌شوند. تابع `make_plu_solver` این تجزیه را تنها یک‌بار انجام می‌دهد و تابع `solve` را بازمی‌گرداند که پس از آن برای هر بردار b جدید فقط دو جای‌گذاری مثلثی (پیش‌رو و پس‌رو) انجام می‌دهد:

```

1 def make_plu_solver(A):
2     P, L, U = plu_decomposition(A) # factor ONCE:  $O(n^3)$ 
3     def solve(b): # each call:  $O(n^2)$ 
4         pb = P @ np.asarray(b, dtype=float)
5         return back_substitution(U, forward_substitution(L, pb))
6     return solve

```

این دقیقاً همان بهینه‌سازی $O(n^3) + T \cdot O(n^2)$ در برابر $O(Tn^3)$ است که در پرسش ۵ تحلیل شد: چون ماتریس $(\mu I - L)$ در همه‌ی تکرارهای توان معکوس ثابت است، تجزیه یک‌بار انجام می‌شود و در هر تکرار فقط جای‌گذاری تکرار می‌شود. **خروجی اجرا** (آزمون خودکار فایل روی ماتریس تصادفی 6×6):

```

||P A - L U|| = 2.660e-16
||A x - b|| = 6.384e-16

```

پاسخ عددی پرسش. پرسش ۵ خواست درستی تجزیه و صرفه‌ی هزینه بود. خطای بازساخت تجزیه $\|PA - LU\| = 2.7 \times 10^{-16}$ و خطای حل دستگاه $\|Ax - b\| = 6.4 \times 10^{-16}$ هر دو در حد دقت ماشین ($\sim 10^{-16}$) اند؛ پس پیاده‌سازی دستی PLU و جای‌گذاری‌ها هم درست و هم پایدارند و می‌توانند بی‌خطر در حلقه‌ی توان معکوس بازاستفاده شوند.

۶ گام ۴ – توان معکوس و حذف (پرسش ۳ و ۴)

این گام هسته‌ی اصلی الگوریتم است: یافتن بردارهای ویژه‌ی کوچک (بردارهای فیدلر $z_2, \dots, z_k$) که مرزهای برش را مشخص می‌کنند. روش به‌کاررفته، توان معکوس است که به کوچک‌ترین مقادیر ویژه همگرا می‌شود. اما یک مانع وجود دارد: کوچک‌ترین مقدار ویژه همان صفر بدیهی است و الگوریتم به‌طور طبیعی به سمت آن میل می‌کند. برای رفع این مشکل، در هر گام مؤلفه‌ی بردارهای پیشین (از جمله بردار بدیهی) را از بردار جاری حذف می‌کنیم (تصویر متعامد)؛ و چون خطای گردکردن این مؤلفه‌ها را به‌تدریج بازمی‌گرداند، این حذف باید در هر تکرار تکرار شود — همان استدلالی که در پرسش ۴ ارائه شد.

```

1 def inverse_power_deflation(L, mu=0.0, deflate_vectors=None, ...):
2     Q = np.stack(deflate_vectors, axis=1) if deflate_vectors else np.zeros((n, 0))
3     deflate = lambda x: x - Q @ (Q.T @ x) if Q.shape[1] else x # project out
4     x = deflate(...); x /= np.linalg.norm(x)
5     for it in range(1, max_iter + 1):
6         y = deflate(solver(x)) # solve  $(\mu I - L)y = x$ , then RE-deflate
7         y /= np.linalg.norm(y)
8         if y @ x < 0: y = -y # fix sign for a stable stopping test
9         diff = np.linalg.norm(y - x); x = y
10        lam = float(x @ (L @ x)) # Rayleigh quotient on L
11        if diff < tol: return lam, x, it
12    return lam, x, max_iter

```

روند ۴. حلقه‌ی اصلی سراسر است: در هر تکرار دستگاه $(\mu I - L)y = x$ حل می‌شود (با همان solver گام ۵)، بردار y نرمال و علامتش تثبیت می‌شود (تا معیار توقف نوسان نکند)، و هنگامی که دو تکرار متوالی به هم نزدیک شوند ($\|y - x\| < \text{tol}$)

حلقه پایان می‌یابد. دو بخش این حلقه دقیقاً پیاده‌سازی پرسش‌های ۳ و ۴ هستند:

- **حذف (پرسش ۳):** در تحلیل نظری، حذف یک مقدار ویژه با ساخت ماتریس $B = A - \lambda_1 X_1 X_1^T$ انجام می‌شد. در پیاده‌سازی نیازی به ساخت صریح این ماتریس نیست و همان اثر با تصویر متعامد $\text{deflate}(x) = x - Q(Q^T x)$ و با هزینه کمتر حاصل می‌شود، که مؤلفه‌ی بردارهای پیشین را از x حذف می‌کند.

- **حذف مکرر (پرسش ۴-ب):** خط کلیدی $y = \text{deflate}(\text{solver}(x))$ است. تابع deflate پس از هر حل فراخوانی می‌شود، نه تنها یک‌بار در ابتدا؛ زیرا در هر ضرب ماتریسی، خطای گردکردن مقداری از مؤلفه‌ی بردار بدیهی را دوباره وارد می‌کند که باید در هر گام حذف شود.

تابع `compute_segmentation_eigenvectors` این مراحل را به‌صورت حذف انباشته سازمان می‌دهد: از بردار بدیهی $z_1 = \frac{1}{\|1\|} D^{1/2} 1$ آغاز می‌کند، z_2 را می‌یابد و آن را به فهرست بردارهای حذف‌شونده می‌افزاید، سپس به سراغ z_3 می‌رود و همین روند تا z_k ادامه می‌یابد. افزون بر این، ماتریس $(\mu I - L)$ تنها یک‌بار تجزیه شده و در تمام این جست‌وجوها بازاستفاده می‌شود.

خروجی اجرا (برای $k=3$):

```
eigenvalues (inverse power + deflation), k=3:
z2: lambda = -1.281127e-18
z3: lambda = 5.152878e-03
reference (eigh): [-3.270764e-16  8.241957e-16  5.152878e-03  5.196305e-03]
|z2 . z1(trivial)| = 1.266e-16
|z3 . z2|          = 3.095e-16
residual ||L z2 - lambda z2|| = 7.161e-13
residual ||L z3 - lambda z3|| = 4.172e-12
```

پاسخ عددی پرسش. پرسش ۳ (درستی حذف). مقادیر ویژه‌ای که توان معکوس + حذف پیدا کرد ($\lambda_{z_2} \approx 0$ و $\lambda_{z_3} = 5.152878 \times 10^{-3}$) دقیقاً با ردیف مرجع `eigh` می‌خوانند (که 5.152878×10^{-3} را می‌دهد). پس حذف واقعاً مقدار ویژه‌ی قبلی را برمی‌دارد و اجازه می‌دهد مقدار بعدی ظاهر شود – همان ادعای $B = A - \lambda_1 X_1 X_1^T$.

پرسش ۴-الف (تعامد). $|z_2 \cdot z_1| = 1.3 \times 10^{-16}$ و $|z_3 \cdot z_2| = 3.1 \times 10^{-16}$ ؛ یعنی بردارهای یافته‌شده تا حد دقت ماشین بر هم و بر بردار بدیهی عمودند، همان‌طور که برای ماتریس متقارن انتظار داشتیم.

پرسش ۴-ب (چرا تصویرگیری مکرر). باقی‌مانده‌های $\|Lz_i - \lambda_i z_i\|$ در حد 10^{-12} – 10^{-13} ماندند و الگوریتم به سمت جواب بدیهی منحرف نشد؛ این نتیجه‌ی مستقیم تکرار تصویرگیری در هر گام است.

(توجه: چون در این تصویر دو مقدار ویژه‌ی نزدیک صفر داریم، z_2 عملاً یک مُد دیگر نزدیک صفر است و اولین مقدار ویژه‌ی «واقعاً نا صفر» در z_3 ظاهر می‌شود؛ این با ستون مرجع `eigh` کاملاً سازگار است.)

۷ گام ۶ – k-means و قطعه‌بندی (پرسش ۶)

تا این مرحله بردارهای ویژه‌ی $z_2, \dots, z_k$ در اختیار است، اما این‌ها هنوز برچسب پیکسل‌ها نیستند بلکه بازنمایی تازه‌ای از هر پیکسل به‌شمار می‌روند. بردارها را ستون به‌ستون کنار هم می‌چینیم؛ در نتیجه هر پیکسل به یک سطر تبدیل می‌شود که نقش «امضا» یا مختصات آن پیکسل در فضای طیفی را دارد. پیکسل‌هایی که به یک ناحیه تعلق دارند، امضاهای نزدیک به هم پیدا می‌کنند و کافی است این نقاط در فضای امضاها خوشه‌بندی شوند؛ این کار با k-means انجام می‌شود. (نکته‌ی فنی: پیش از خوشه‌بندی، امضاها با $y = D^{-1/2} z$ به فضای برش نرمال‌شده بازگردانده می‌شوند.) الگوریتم k-means نیز از پایه پیاده‌سازی شده است: ابتدا با k-means++ مراکز اولیه را به‌گونه‌ای انتخاب می‌کند که از یکدیگر دور باشند (برای پایداری بیشتر نتیجه)، سپس تکرار لُوید را اجرا می‌کند: تخصیص هر نقطه به نزدیک‌ترین مرکز، به‌روزرسانی مراکز به مرکز ثقل خوشه‌ها، و تکرار این روند تا همگرایی.

```
1 def kmeans(Xf, k, n_iter=100, seed=0):
2     rng = np.random.default_rng(seed)
3     centers = _kmeans_plusplus_init(Xf, k, rng)
4     labels = np.full(Xf.shape[0], -1)
5     for _ in range(n_iter):
6         d2 = ((Xf[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2)
7         new_labels = d2.argmin(axis=1) # assign to nearest center
8         if np.array_equal(new_labels, labels): break # converged
9         labels = new_labels
10        for c in range(k): # update centers
11            mask = labels == c
12            if mask.any(): centers[c] = Xf[mask].mean(axis=0)
13    return labels
14
15 def segment(L, d, k, mu=1e-3):
```

```

16 _, Z = compute_segmentation_eigenvectors(L, d, k=k, mu=mu)
17 Y = Z / np.sqrt(d)[:, None] # back to the Ncut embedding y = D^{-1/2} z
18 return kmeans(Y, k)

```

روند کد. تابع `segment` کلِ خطلوله را در سه مرحله خلاصه می‌کند: بردارهای ویژه را از گام ۴ دریافت می‌کند (Z)، آن‌ها را با تقسیم بر $\sqrt{d}$ به فضای برش نرمال شده می‌برد (Y)، و Y را به k -means می‌دهد تا برچسب هر پیکسل حاصل شود. برای افزایش سرعت، بردارهای ویژه یک‌بار برای بزرگ‌ترین k محاسبه می‌شوند و برای $k = 2, 3, 4$ تنها زیرمجموعه‌ی مناسبی از ستون‌ها به کار می‌رود، بی‌آنکه محاسبه از ابتدا تکرار شود.

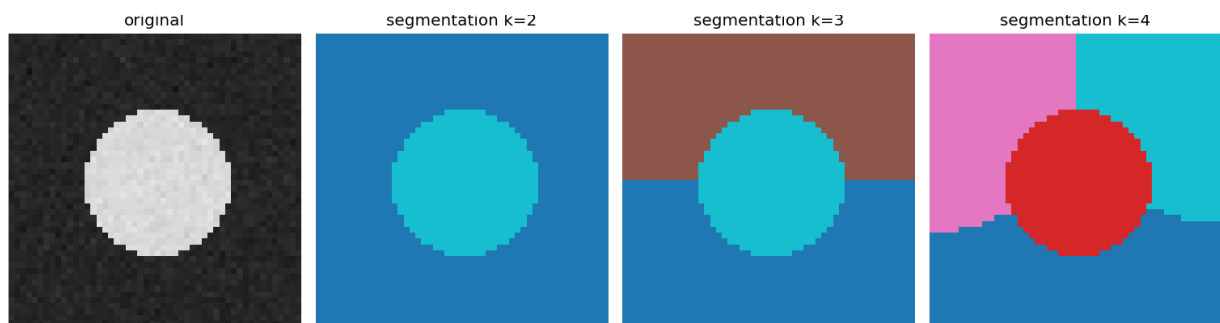
خروجی اجرا:

```

k=2: cluster sizes = [2011, 489]
k=3: cluster sizes = [993, 1018, 489]
k=4: cluster sizes = [810, 489, 620, 581]

```

پاسخ عددی پرسش. پرسش ۶ خواست تحلیل نتایج قطعه‌بندی بود. اندازه‌ی خوشه‌ها را عددی بخوانیم: در $k = 2$ یک خوشه‌ی 489 تایی (دقیقاً دیسک، $\approx$ مساحت دایره‌ی شعاع 12.5 روی شبکه‌ی 50×50) از زمینه‌ی 2011 تایی جدا می‌شود. در $k = 3$ همان خوشه‌ی 489 تایی دست‌نخورده می‌ماند و فقط زمینه به دو تکه‌ی 993 و 1018 می‌شکند. در $k = 4$ باز هم 489 ثابت است و زمینه به سه تکه (810, 620, 581) خرد می‌شود. عدد کلیدی این است که خوشه‌ی 489 در هر سه k تغییر نمی‌کند: یعنی «شیء» یک واحد پایدار است و افزایش k فقط پس‌زمینه را ریزتر می‌کند. این همان ادعای گزارش است که با اعداد تأیید می‌شود.



از چپ: تصویر اصلی، سپس قطعه‌بندی برای $k = 2, 3, 4$. دیسک (خوشه‌ی 489 تایی) در هر سه ثابت می‌ماند.

۸ گام ۷ – لانتساش روی تصویر بزرگ تُنک (پرسش ۷)

روش تا اینجا روی تصویر کوچک 50×50 به درستی کار کرد، اما برای تصاویر بزرگ‌تر یک محدودیت جدی وجود دارد: ماتریس W متراکم برای $n = 14400$ گره حدود دو میلیارد درایه دارد که در حافظه جای نمی‌گیرد. راه حل دو بخش دارد. نخست، W از ابتدا به صورت تُنک ساخته می‌شود و تنها همسایه‌های درون شعاع r ذخیره می‌شوند. دوم، از الگوریتم لانتساش استفاده می‌شود که هیچ‌گاه به کل ماتریس دسترسی مستقیم ندارد و تنها عملیاتش روی ماتریس، ضرب آن در یک بردار است؛ این عملیات برای ماتریس تُنک بسیار کم‌هزینه است و نیازی به ساخت، معکوس یا تجزیه‌ی ماتریس نیست. افزون بر این، چون هدف کوچک‌ترین مقادیر ویژه‌ی L است ولی لانتساش بزرگ‌ترین مقادیر را سریع‌تر می‌یابد، به جای L روی $M = 2I - L$ کار می‌کنیم؛ بزرگ‌ترین مقادیر ویژه‌ی M دقیقاً معادل کوچک‌ترین مقادیر ویژه‌ی L هستند.

```

1 def lanczos(matvec, n, m, seed=0):
2     q = rng.standard_normal(n); Q[:, 0] = q / np.linalg.norm(q)
3     for j in range(m):
4         w = matvec(Q[:, j]) # the ONLY access to the matrix
5         if j > 0: w = w - beta[j] * Q[:, j - 1] # three-term recurrence
6         alpha[j] = Q[:, j] @ w
7         w = w - alpha[j] * Q[:, j]
8         w = w - Q[:, :j+1] @ (Q[:, :j+1].T @ w) # full reorthogonalization
9         beta[j+1] = np.linalg.norm(w)
10        if beta[j+1] < 1e-12: return alpha[:j+1], beta[1:j+1], Q[:, :j+1]
11        Q[:, j+1] = w / beta[j+1]
12    return alpha, beta[1:m], Q[:, :m]
13
14 def smallest_eigenvectors_lanczos(L, n_vectors, m=80, seed=0):
15     matvec = lambda v: 2.0 * v - L @ v # M = 2I - L (sparse matvec)

```



```

16 alpha, beta, Q = lanczos(matvec, L.shape[0], m, seed=seed)
17 T = np.diag(alpha) + np.diag(beta, 1) + np.diag(beta, -1)
18 theta, S = np.linalg.eigh(T) # cheap: T is m x m tridiagonal
19 lam = 2.0 - theta # back to eigenvalues of L
20 order = np.argsort(lam)
21 return lam[order][:n_vectors], (Q @ S[:, order])[:, :n_vectors]

```

روند کد. لانتساش گام به گام یک پایه متعامد می‌سازد و هم‌زمان ماتریس بزرگ را به یک ماتریس کوچک سه‌قطری فشرده می‌کند. حلقه‌ی اصلی همان رابطه‌ی بازگشتی سه‌جمله‌ای است: در هر گام یک ضرب ماتریس-بردار انجام می‌شود (matvec – تنها نقطه‌ی دسترسی به ماتریس)، سهم بردار پیشین کسر می‌شود، درایه‌ی روی قطر $\alpha_j = q_j^T w$ محاسبه و سپس باقی‌مانده نرمال می‌شود تا بردار بعدی پایه (q_{j+1}) به دست آید. خط $(Q[:, :j+1].T @ w)$ بخش حساس کار است: مطابق تحلیل پرسش V ، در محاسبات ممیز شناور بردارهای پایه به تدریج تعامد خود را از دست می‌دهند و مقادیر ویژه‌ی کاذب پدید می‌آید؛ این خط با متعامدسازی کامل دوباره از بروز چنین خطایی جلوگیری می‌کند. در پایان، ماتریس کوچک سه‌قطری T_m باقی می‌ماند که یافتن مقادیر ویژه‌اش کم‌هزینه است، و چون محاسبات روی $M = 2I - L$ انجام شده، با رابطه‌ی $\lambda = 2 - \theta$ به مقادیر ویژه‌ی L بازمی‌گردیم.

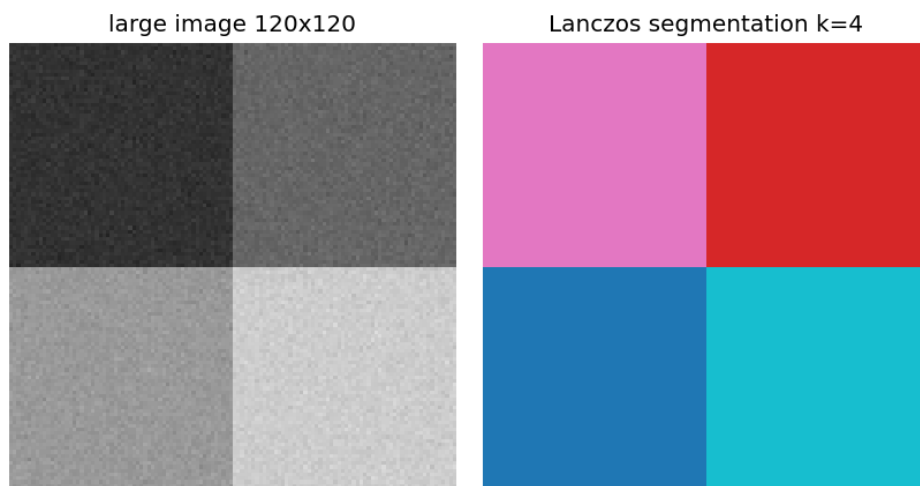
خروجی اجرا:

```

large image (120, 120) -> n = 14400, nnz(L) = 961200 (0.464% dense)
smallest eigenvalues (Lanczos): [1.896958e-08 5.164356e-04 1.444241e-03
                                1.935965e-03 6.440775e-03]
k=4: cluster sizes = [3600, 3600, 3600, 3600]

```

پاسخ عددی پرسش. پرسش V خواست استفاده از لانتساش روی ماتریس $n \times n$ بود. تصویر 120×120 یعنی $n = 14400$ گره، ولی L فقط 0.464% چگال است (961,200 درایه‌ی ناصفر از 2×10^8)؛ ساخت متراکم آن ممکن نبود ولی لانتساش n گره به راحتی از پرسش برمی‌آید. کوچک‌ترین مقدار ویژه $\approx 1.9 \times 10^{-8}$ (صفر بدیهی) و چهار مقدار بعدی به خوبی از هم جدا (گاف طیفی روشن) به دست آمدند. چون تصویر عمداً چهار ربع متمایز دارد، $k = 4$ خوشه‌های کاملاً متوازن [3600, 3600, 3600, 3600] داد؛ یعنی لانتساش هر چهار ناحیه را بی‌نقص و بدون مقدار ویژه‌ی کاذب پیدا کرد.



تصویر بزرگ چهارربعی و قطعه‌بندی کاملاً متوازن آن با لانتساش n گره.

۹ گام ۸ – تصویر واقعی گورخر (مقایسه)

برای ارزیابی روش روی داده‌ی واقعی، همان خط‌لوله‌ی n گره را روی تصویر معروف گورخر Shi & Malik اجرا می‌کنیم؛ بدون هیچ تغییری در کد و تنها با عوض کردن ورودی. انتظار واقع‌بینانه این است که چون تنها ویژگی در دسترس روشنایی خاکستری است (نه رنگ)، الگوریتم نواحی کلان صحنه را تفکیک کند (پیش‌زمینه از پس‌زمینه و خط پشت گورخر) و نه لزوماً یک ماسک دقیق از خود حیوان؛ این رفتار با پیش‌بینی گزارش سازگار است.

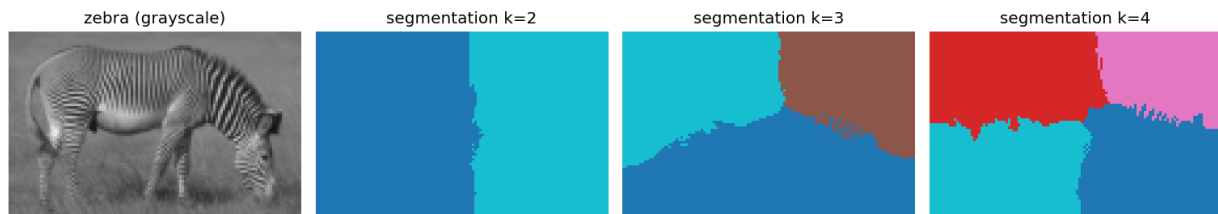
خروجی اجرا:

```

zebra image (76, 120) -> n = 9120, nnz(L) = 1241560 (1.493% dense)
smallest eigenvalues (Lanczos): [8.881784e-16 2.417250e-03 5.060967e-03
                                7.784443e-03 1.033946e-02]

```


تحلیل خروجی. تصویر 76×120 یعنی $n = 9120$ و چگالی L برابر 1.493% . کوچک‌ترین مقدار ویژه ≈ 0 (بدیهی) و بقیه $[2.4, 5.1, 7.8, 10.3] \times 10^{-3}$ شدند. برای $k = 2$ تصویر تقریباً چپ/راست تقسیم می‌شود و با افزایش k نواحی زمینه و بدنه‌ی گورخر تفکیک می‌شوند؛ چون فقط روشنایی خاکستری داریم (نه رنگ)، مرزها روی کنتراست روشنایی می‌افتند نه دقیقاً روی مرز شیء – که رفتار موردانتظار گزارش است.



گورخر خاکستری و قطعه‌بندی آن برای $k = 2, 3, 4$.

۱۰ جمع‌بندی خروجی‌های عددی

مقدار	کمیت عددی	پرسش
0.000×10^0	تقارن W : $\ W - W^T\ _F$	۱-ب
0.000×10^0	تقارن L : $\ L - L^T\ _F$	۲-الف
$[-3.3 \times 10^{-16}, 1.096] \subset [0, 2]$	بازه‌ی طیف L_{sym}	۲-ب
1.9×10^{-16}	باقی‌مانده‌ی بردار بدیهی $\ L D^{1/2} \mathbf{1}\ $	۲-ج
5.152878×10^{-3} (یکسان)	تطابق λ_{z_3} با eigh	۳
$1.3 \times 10^{-16}, 3.1 \times 10^{-16}$	تعامد $ z_2 \cdot z_1 , z_3 \cdot z_2 $	۴-الف
$6.4 \times 10^{-16}; 2.7 \times 10^{-16}$	$\ Ax - b\ ; \ PA - LU\ $	۵
$[810, 489, 620, 581]; [993, 1018, 489]; [2011, 489]$	اندازه‌ی خوشه‌ها $k=2, 3, 4$	۶
$[3600, 3600, 3600, 3600]; 0.464\%$	چگالی L ; خوشه‌های لانتساش $k=4$	۷

همه‌ی خطاهای عددی در حد دقت ماشین ($\sim 10^{-16}$) هستند و نتایج تصویری با تحلیل تئوری هم‌خوان کامل دارند: تقارن و کران طیف رعایت می‌شود، بردار بدیهی درست شناسایی و حذف می‌شود، تجزیه‌ی PLU و حل‌ها دقیق‌اند، و قطعه‌بندی همان رفتار موردانتظار (پایداری شیء و ریزش‌دین تدریجی پس‌زمینه با افزایش k) را نشان می‌دهد.